Handoff: Lavish-Based Branch Review Skill

Goal

Build a local AI-assisted branch review skill that generates an interactive HTML review cockpit for a Git branch diff and uses Lavish-AXI as the human-agent feedback bridge.

The tool should help a human reviewer understand AI-generated or human-generated code changes by presenting:

- What changed
- Why it matters
- Which files are important
- What behavior may have changed
- Which areas are risky
- What tests should be checked or added
- What suspiciously did not change
- A visual diff and optional diagrams
- An interactive way to ask questions and challenge the review

The intended user experience is:

```
User runs a skill/command in the repository
→ skill compares current branch with main/develop
→ skill generates review.html
→ skill opens it with lavish-axi
→ user reviews the presentation in browser
→ user asks questions or annotates sections
→ agent polls Lavish feedback
→ agent answers back into Lavish/browser
```

This is not intended to replace human review. The goal is to reduce review navigation cost and make AI-generated code easier to audit.

High-Level Architecture

```
Claude/Codex/Agent Skill
        |
        | invokes
        v
Git Diff Collector Script
        |
```

```
        | writes
        v
.review-agent/
  context.json
  diff.patch
  diff-stat.txt
  changed-files.json
  commits.txt
  analysis.json
  review.html
        |
        | opens
        v
Lavish-AXI Local Browser UI
        |
        | human feedback / questions / annotations
        v
lavish-axi poll
        |
        | agent receives questions
        v
Agent answers and sends reply back
```

The architecture intentionally avoids MCP because some target environments prohibit MCP servers.

Lavish-AXI is used as the local browser interaction layer. The custom skill is responsible for Git analysis and generating the code-review-specific HTML artifact.

Main Components

1. Agent Skill

Suggested location:

```
.claude/skills/branch-review-cockpit/
  SKILL.md
  scripts/
    collect_review_context.py
    generate_review_html.py
  templates/
    review.html.j2
  assets/
```

```
    style.css
    app.js
```

The skill should support commands such as:

```
/review-branch main
/review-branch develop
/review-poll
/review-answer
/review-close
```

The skill is responsible for:

1. Collecting branch diff context.
2. Asking the agent to analyze the diff.
3. Generating a structured HTML review artifact.
4. Opening the artifact with Lavish-AXI.
5. Polling user questions and annotations.
6. Sending answers back to Lavish.

---

2. Git Diff Collector

The collector should compare the current branch against a base branch.

Example logic:

```
BASE_BRANCH="${1:-main}"
MERGE_BASE=$(git merge-base HEAD "$BASE_BRANCH")

git diff "$MERGE_BASE"...HEAD > .review-agent/diff.patch
git diff --stat "$MERGE_BASE"...HEAD > .review-agent/diff-stat.txt
git diff --name-status "$MERGE_BASE"...HEAD > .review-agent/name-status.txt
git log --oneline "$MERGE_BASE"..HEAD > .review-agent/commits.txt
```

The collector should also generate a structured `context.json`.

Example:

```
{
  "base_branch": "main",
  "current_branch": "feature/example",
  "merge_base": "abc123",
```

```
  "head": "def456",
  "changed_files": [
    {
      "path": "src/release_runner.cpp",
      "status": "modified",
      "language": "cpp"
    }
  ],
  "summary": {
    "files_changed": 6,
    "insertions": 320,
    "deletions": 80
  }
}
```

---

3. Agent Analysis Model

The agent should transform raw diff context into structured review data.

Suggested output file:

```
.review-agent/analysis.json
```

Suggested schema:

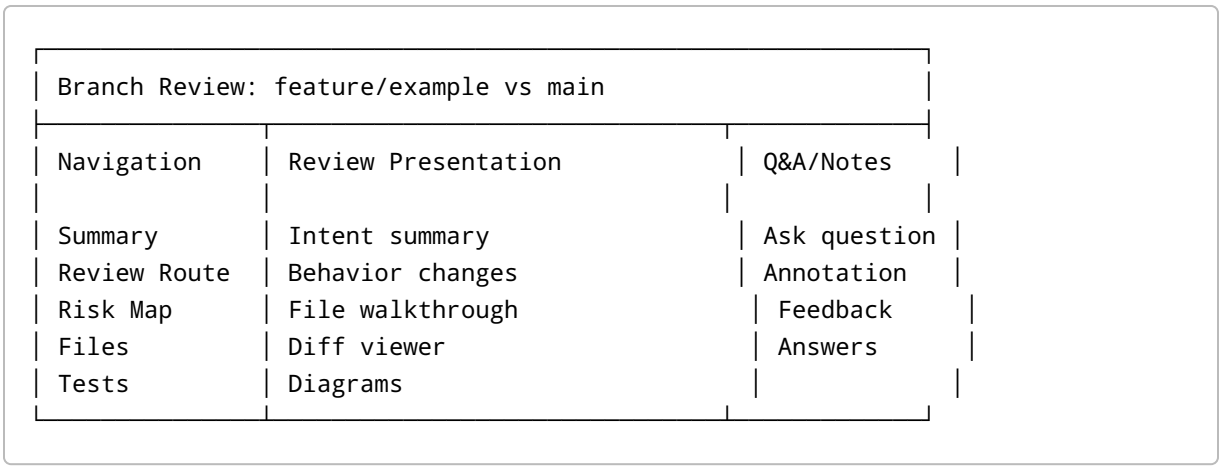
```
{
  "title": "Branch Review: feature/example vs main",
  "intent_summary": "Short explanation of what the branch appears to do.",
  "behavior_changes": [
    "External or user-visible behavior change #1",
    "External or user-visible behavior change #2"
  ],
  "review_route": [
    {
      "step": 1,
      "title": "Start with orchestrator changes",
      "reason": "Core behavior is changed here.",
      "files": ["src/orchestrator/release_runner.cpp"]
    }
  ],
  "risk_map": [
    {
      "level": "high",
```

```
      "area": "Retry behavior",
      "reason": "Retries may duplicate side effects if job creation is not
idempotent.",
      "files": ["src/orchestrator/release_runner.cpp"],
      "questions": [
        "Is job creation idempotent?",
        "What happens if cancellation occurs during retry?"
      ]
    }
  ],
  "file_walkthrough": [
    {
      "path": "src/orchestrator/release_runner.cpp",
      "role": "Core behavior change",
      "what_changed": "Retry loop added around build execution.",
      "review_focus": [
        "Check retry boundaries.",
        "Check cancellation behavior.",
        "Check logging and error propagation."
      ]
    }
  ],
  "suspicious_omissions": [
    {
      "area": "Tests",
      "reason": "Core behavior changed but no tests were updated."
    }
  ],
  "test_checklist": [
    "Retry after transient failure.",
    "No retry after user cancellation.",
    "No duplicate job creation."
  ],
  "diagrams": [
    {
      "title": "New retry flow",
      "type": "mermaid",
      "source": "sequenceDiagram\n  participant User\n  participant
Orchestrator\n  participant Worker\n  User->>Orchestrator: Start\n
Orchestrator->>Worker: Run\n  Worker-->>Orchestrator: Failure\n  Orchestrator-
>>Worker: Retry once"
    }
  ]
}
```

HTML Review Cockpit

The generated `review.html` should be self-contained or mostly self-contained.

Suggested layout:



Core sections:

1. **Executive Summary**
2. What the branch appears to do.
3. Whether the diff looks mechanical, behavioral, risky, or incomplete.
4. **Review Route**
5. Ordered path for human review.
6. “Start here, then inspect these files, then verify tests.”
7. **Behavior Changes**
8. Human-readable description of what behavior may change.
9. **Risk Map**
10. Group risks by correctness, compatibility, concurrency, security, performance, maintainability, and test coverage.
11. **File Walkthrough**

12. For each changed file:
    - Role in the change
    - What changed
    - Why it matters
    - What to inspect manually
13. **Diff Viewer**
14. Render the actual diff.
15. Prefer side-by-side when possible.
16. Fall back to unified diff for simplicity.
17. **Suspicious Omissions**
18. Tests not changed.
19. Docs not changed.
20. Config not changed.
21. Callers not updated.
22. Error handling not updated.
23. **Test Checklist**
24. Concrete tests to run or add.
25. **Diagrams**
26. Mermaid sequence diagrams, state diagrams, or flow diagrams where useful.
27. **Interactive Q&A**
28. Handled by Lavish annotations and chat feedback rather than custom browser/server logic.

Lavish-AXI Integration

The skill should generate the HTML artifact and open it with Lavish-AXI.

Example:

```
npx -y lavish-axi .review-agent/review.html
```

The agent should then poll for feedback:

```
npx -y lavish-axi poll .review-agent/review.html
```

When the user asks a question or annotates a section, the agent receives the feedback from `poll`.

The agent should answer in the normal agent chat and also send the reply back to the browser:

```
npx -y lavish-axi poll .review-agent/review.html --agent-reply "Answer text here"
```

Suggested skill commands:

```
/review-branch main
```

Collect context, generate review, open Lavish.

```
/review-poll
```

Poll Lavish for new questions or annotations.

```
/review-answer
```

Poll feedback, answer pending questions, and send replies back.

```
/review-close
```

Close or stop the Lavish session if needed.

Suggested MVP Flow

MVP 1: Static Review Artifact

Goal: generate useful review HTML, no live Q&A yet.

Steps:

1. Collect branch diff.
2. Ask agent to generate `analysis.json`.
3. Render `review.html`.
4. Open with Lavish.
5. User reviews manually.

Success criteria:

- The HTML makes the diff easier to review than GitHub/GitLab alone.
- The review route identifies the most important files.
- Risks and test gaps are useful enough to guide manual inspection.

MVP 2: Lavish Feedback Loop

Goal: allow user to ask questions from the browser.

Steps:

1. User asks questions or annotates HTML in Lavish.
2. Agent runs `lavish-axi poll`.
3. Agent answers question.
4. Agent sends reply back to Lavish.

Success criteria:

- User can challenge a risk or file-level comment.
- Agent can answer using branch diff and repository context.
- The answer appears both in agent chat and browser UI.

MVP 3: Better Diff and Diagrams

Goal: improve presentation quality.

Possible improvements:

- Integrate `diff2html` for side-by-side diff rendering.
- Add Mermaid diagrams for flows.
- Add collapsible file sections.
- Add risk badges.
- Add “review first” labels.
- Add links from risk items to diff sections.

MVP 4: Optional Local Python Fallback

Goal: support environments where `npx` or Node execution is blocked.

Implement a small Python Lavish-like fallback:

```
review-axi open review.html
review-axi poll review.html
review-axi reply review.html --id q_001 --file answer.md
review-axi close review.html
```

This is not required for the first version.

Security and Environment Constraints

The tool should be safe for corporate repositories.

Required constraints:

- No MCP server dependency.
- No remote upload of repository code unless explicitly configured.
- No arbitrary shell command execution from browser input.
- No browser endpoint that can read arbitrary local files.
- Prefer local assets over CDN.
- Keep all generated files under `.review-agent/`.
- Add `.review-agent/` to `.gitignore`.
- Treat Lavish/browser feedback as untrusted text.
- Do not automatically apply code changes from the review artifact.
- Do not automatically commit anything.

Recommended `.gitignore` entry:

```
.review-agent/
```

Non-Goals

The first version should not try to:

- Replace GitHub/GitLab PR review.
- Automatically approve or reject code.
- Automatically rewrite the branch.

- Run as a long-lived background service.
- Depend on MCP.
- Require a dedicated SaaS backend.
- Require direct browser-to-LLM communication.
- Support every language perfectly.
- Build a full IDE.

The goal is a focused local review cockpit.

Open Questions

1. Target Agent Runtime

Which agent environment should be supported first?

Options:

- Claude Code skill
- Codex CLI prompt/script
- Generic shell script usable by any agent
- Repository-local command with no agent-specific assumptions

Recommendation: start with Claude Code-style skill layout, but keep scripts generic.

---

2. Base Branch Detection

Should the skill require the user to specify the base branch, or auto-detect?

Options:

```
/review-branch main
/review-branch develop
/review-branch auto
```

Recommendation: support explicit base first. Add auto-detection later.

---

3. Diff Size Limits

How should the tool handle huge diffs?

Possible strategies:

- Warn and continue.
- Summarize only changed file list and stats.
- Exclude generated files.
- Exclude lock files.
- Exclude vendored code.
- Ask user to split branch.

Recommendation: implement basic excludes first: `node_modules`, `vendor`, generated files, lock files, build artifacts.

---

4. Repository Context Depth

How much code beyond the diff should the agent inspect?

Options:

- Diff only.
- Diff + nearby functions.
- Diff + callers/callees found via ripgrep.
- Diff + project-specific review rules.

Recommendation: start with diff only, then add nearby function context for selected high-risk files.

---

5. Language Awareness

Should the first version be language-neutral or optimized for C++/Python/TypeScript?

Recommendation: language-neutral first. Add optional language-specific heuristics later.

Examples:

- C++: ownership, lifetime, threading, ABI, error propagation.
- Python: async behavior, typing, exceptions, packaging.
- TypeScript: type narrowing, API contracts, runtime validation.

---

6. Test Integration

Should the tool run tests automatically?

Options:

- No test execution.

- Detect available test commands.
- Ask user before running tests.
- Run only safe commands configured in a repo file.

Recommendation: do not run tests automatically in MVP. Generate a checklist first.

---

7. Review Configuration

Should repositories define their own review policy?

Possible config file:

```
.review-agent.yaml
```

Example:

```
base_branch: develop
exclude:
  - package-lock.json
  - "*.generated.cpp"
focus:
  - correctness
  - compatibility
  - tests
  - security
language_hints:
  - cpp
  - python
```

Recommendation: add config after MVP.

---

8. HTML Asset Strategy

Should assets be local or CDN?

Recommendation: local vendored assets only, especially for corporate environments.

---

9. Lavish Dependency Policy

Is `npx -y lavish-axi` acceptable in target environments?

Options:

- Use `npx -y lavish-axi`.
- Pin a Lavish version.
- Vendor Lavish dependency.
- Provide Python fallback.

Recommendation: start with pinned Lavish version if possible. Add Python fallback only if needed.

---

10. Answer Persistence

Should Q&A answers be written back into the HTML artifact?

Options:

- Lavish session only.
- Separate `qa.jsonl`.
- Regenerate HTML with Q&A history.
- Export final review as Markdown.

Recommendation: write Q&A to `.review-agent/qa.jsonl` and optionally regenerate the HTML.

Suggested First Implementation Milestone

Build the smallest useful version:

```
/review-branch main
```

Does:

1. Creates `.review-agent/`.
2. Collects Git diff against `main`.
3. Generates `context.json`, `diff.patch`, `diff-stat.txt`.
4. Agent produces `analysis.json`.
5. Renders `review.html`.
6. Opens with Lavish-AXI.
7. Supports polling and replying to user questions through Lavish.

The first version can use a simple HTML template and unified diff. Better diff rendering and diagrams can come later.

Success Criteria

The tool is successful if a human reviewer can answer these questions faster than with a plain diff:

1. What is the branch trying to do?
2. Which files should I inspect first?
3. What behavior changed?
4. What are the highest-risk changes?
5. What assumptions should I challenge?
6. What tests should I run or add?
7. What important files did not change but maybe should have?
8. Can I ask “why did you say this is risky?” and get a useful answer?

The review cockpit should make human review more focused, not more automated.